Maharaja Ranjit Singh Punjab Technical University

Smart Attendance System Project Specifications

Welcome to the official technical documentation of the **Smart University Attendance System**. This system is an state-of-the-art, AI-powered classroom attendance automation platform designed to eliminate manual proxies, provide high-precision real-time student face identification, and generate administrative analytics.

> [!NOTE]

> This system leverages browser-side GPU-accelerated deep learning via `face-api.js` (loaded via CDN) to process camera streams locally. This architecture ensures high-performance face recognition and scaling without expensive backend GPU server infrastructure.

■■ System Architecture Diagram

```
graph TD
    subgraph Client-Side (React Vite SPA)
        Student[Student Portal] -->|Webcam / Face Upload| FaceEnrollment[Face Enrollment Portal]
        Teacher[Teacher Portal] -->|Group Photo/Live Scan| FaceAPI[Local Face-API.js Neural Engine]
        Admin[Master Admin / HOD] -->|CRUD & Normalization| AdminPanel[Admin Dashboard]
    end

    subgraph Backend APIs (Node.js & Express)
        Endpoints[REST Endpoints /api/*] -->|Request Validation| GlobalHandler[Exception & Middleware Layer]
        BodyParser[Body Parser 50MB] -->|Base64 Streams| Endpoints
    end

    subgraph Database Layer (MongoDB Atlas)
        Mongoose[Mongoose Schemas] -->|Indexes & Operations| MongoDB[(MongoDB Cloud Collections)]
    end

    FaceAPI -->|Base64 Enrolled Descriptor| Endpoints
    FaceEnrollment -->|Base64 Face Profile| Endpoints
    AdminPanel -->|Data Control & PDF Reports| Endpoints
    GlobalHandler --> Mongoose
```

■ 1. Project Requirements & Core Objective

The manual tracking of university students is error-prone, time-consuming, and subject to proxy attendance. The Smart University Attendance System replaces traditional roster logs with a camera-driven biometric checkpoint:

- **Automated Verification:** Auto-detects, aligns, and matches faces in live video or static class photos.

- **Tamper-proof Proof of Attendance:** Stores a stamped high-resolution classroom snapshot (watermarked with the date, time, course, semester, and attendance statistics) for every lecture marked.
- **Self-Service Portals:** Individual dashboards for students to track progress and register their profiles, teachers to mark and audit historical records, and admins to manage entities.

■ 2. Complete User Flows

A. Master Administrator Flow

1. **Login Checkpoint:** Signs in via unique credentials (`username` or `email`).
2. **Dashboard Landing:** Enters the *Student Submissions* page directly as the default panel.
3. **Database Administration (CRUD):**

- **Management:** Adds, updates, views, or deletes accounts for **Students**, **Teachers**, **Departments**, and other **Admins**.
- **Custom Photos:** Overwrites profile pictures for any member instantly.

4. **Data Sharing & Audit:**

- **Secure Sharing:** Extracts credentials and sends them to teachers/students via native mobile sharing options or direct clipboard copies.
- **Audit Logs:** Reviews registration timestamps and student batch records.

5. **Data Normalization Routine:** Triggers a system-wide normalization process that dynamically aligns student semesters based on batch years, verifies username enrollment formats, and updates standard password baselines.

B. HOD / Department Admin Flow

1. **Department Isolation:** Logs in as the Head of Department (HOD) for a specific domain (e.g., *Computational Sciences*).
2. **Access Control:** Restricted strictly to students enrolled in courses mapped to their department.
3. **Roster Monitoring:** Reviews teacher registrations and attendance reports within their department.
4. **Attendance Analytics:** Analyzes overall attendance percentages, present/absent curves, and checks for classes lagging in syllabus coverage.

C. Teacher Flow

1. **Dashboard Overview:** Displays teacher metadata, qualifications, and primary assigned subjects.
2. **Attendance Configuration:** Selects Course, Semester, Assigned Subject, and Session (e.g., *Lecture 1*).
3. **Marking Mode Selection:**

- **Manual Mode:** Conventional manual checklist of present/absent indicators.
- **Smart Mode (AI-Driven):** Uses two advanced camera modules:
- *Class Group Analysis:* Captures a wide snapshot of the class. The local neural network detects all faces, matches them against the department database, watermarks the photo, and marks attendance.

- *Continuous Live Scan:* Opens a live video frame. Students walk past the camera; a real-time detector scans them and flashes a success popup showing their profile avatar when marked present.

4. **Attendance Proof Stamping:** Captures a high-resolution proof image and uploads it.
5. **Record Verification & Export:** Generates custom-designed PDF reports complete with MRSPTU header formatting (supporting bilingual English/Punjabi titles) and exports students lists.

6. **Deletion Request Vault:** To delete erroneous submissions, the teacher must supply their password as an authorization key.

D. Student Flow

1. **Identity Setup (First-Time):** Accesses `/upload-face`, enters their enrollment number, starts their web camera, captures their frontal face, and uploads it.
2. **Self-Service Dashboard:** Enters their personalized dashboard to view:

- **Attendance Statistics:** High-fidelity green/yellow/red indicators highlighting total classes, days present, and days absent.
- **Monthly Calendar Visualizer:** Colors dates dynamically (Green for present, Red for absent, Grey for weekends) for easy visual tracking.
- **Credential Management:** Secure password update modal.

■■ 3. Technical Requirements & Stack

The application is structured as a decoupled **MERN (MongoDB, Express, React, Node)** workspace optimized for rapid static assets compilation and micro-payload queries.

| Component | Technology | Rationale & Purpose |

| :--- | :--- | :--- |

| **Frontend Core** | React.js (Vite) | Offers rapid component re-renders and virtual DOM execution for real-time video streaming overlays. |

| **Styling & Theme** | Modern Glassmorphism CSS | Glassy backdrops, smooth radial gradients, hover micro-animations, and fluid responsive grids. |

| **Face AI Engine** | `face-api.js` (Vlad Mandic CDN) | Client-side neural models (SSD Mobilenet v1, Tiny Face, Landmarks, Recognizer) running locally in-browser. |

| **Reporting Engine** | `jspdf` & `jspdf-autotable` | High-fidelity table formatting, direct client-side PDF drawing with MRSPTU headers. |

| **Backend Framework** | Node.js + Express.js | Low-latency REST endpoints with comprehensive route logs and global error boundaries. |

| **Database** | MongoDB Atlas Cloud | Scalable NoSQL storage designed for nesting Base64 image payloads and flexible JSON objects. |

| **DB Client** | Mongoose ORM | Strict schema constraints, hooks, pre-seeding defaults, and query optimization. |

■ 4. AI Face Recognition Deep Dive

The Smart Attendance system integrates a production-grade facial recognition system executing in the client's browser, eliminating CPU bottlenecks on the server.

> [!TIP]

> **Deep Learning Models Loaded:**

> 1. **SSD Mobilenet v1:** High-accuracy Single Shot Multibox Detector utilizing MobileNet backbones for accurate bounding-box calculations. Used for static group snapshots and face enrollment.

> 2. **Tiny Face Detector:** Sub-millisecond light-weight detector optimized for real-time mobile/desktop browser video loops. Used for the Live Scanner.

> 3. **Face Landmark 68 Net:** Maps 68 key points across the jaw, nose, eyes, and eyebrows to perform face alignment, accounting for tilts and yaw changes.

> 4. **Face Recognition Net:** Consists of a ResNet-34 style architecture trained on 3M+ faces. Generates a unique 128-dimensional float array (`Descriptor`) representing facial features.

The Face Matching Pipeline

```
[ Webcam Frame / Image ]
        ■
        ▼
[ SSD Mobilenet / Tiny Detector ] ■■■ Extracts Bounding Boxes
        ■
        ▼
[ Face Landmark 68 Net ] ■■■■■■■■■■■■ Align face geometry
        ■
        ▼
[ Face Recognition Net ] ■■■■■■■■■■■■ Generate 128-Dimensional Vector
        ■
        ▼
[ FaceMatcher (Euclidean Distance) ] ■■■ Compare with database enrollments
        ■
        ■■■■ Distance <= 0.6  ■■■ Match Found! Mark "Present"
        ■■■■ Distance > 0.6   ■■■ Label as "Unknown"
```

■ 5. Security Protocols & Data Integrity Controls

- **Database Pre-Seeding Safeguard:** On startup, the Mongoose engine checks for existing users. If empty, it securely registers the default master administrator account:
- **Username:** `Adminmanminder`
- **Email:** `manminder4313@gmail.com`
- **Default Password Baseline:** Restored via normalized security profiles.

- **Large Payload Support:** `bodyParser.json` and `bodyParser.urlencoded` are configured to a limit of `50mb` to safely capture high-resolution Base64 image payloads (webcam streams, enrolled faces, and group proof stamps).
- **Two-Factor Action Verification:** Attendance deletion is a highly destructive administrative action. The system requires teachers to complete a password validation checkout before records are removed from MongoDB.
- **Data Normalization Portal:** Administrators can automatically clean, align, and sync database schemas (e.g. recalculating student semesters based on entry batches) to prevent corrupted historical stats.

■ 6. Database Collections & Mongoose Schemas

The database structure consists of 5 highly correlated collections configured with strict indexes and validation metrics.

■ A. Admin Schema (`Admin`)

Represents the Master Administrators controlling all departments and data assets.

```
const AdminSchema = new mongoose.Schema({
  username: { type: String, unique: true },     // Custom alphanumeric username
  fullName: { type: String },                   // Administrator's full name
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },   // Direct secure password string
  contact: { type: String },                    // Contact phone number
  profilePhoto: { type: String },               // Base64 compressed user photo
  role: { type: String, default: 'admin' }      // Authorization role
}, { timestamps: true, id: false });
```

■ B. Department Schema (`Department`)

Represents Department Heads (HODs) who can monitor classes and access rosters within their mapped domain.

```
const DepartmentSchema = new mongoose.Schema({
  department: { type: String, required: true },  // Department name (e.g. Computational Scienc
es)
  headName: { type: String, required: true },    // HOD Full name
  email: { type: String, required: true },
  phone: { type: String, required: true },
  profilePhoto: { type: String },                // Base64 profile thumbnail
  username: { type: String },                    // HOD dashboard username login
  password: { type: String },                    // HOD password string
  role: { type: String, default: 'department' }  // Authorization role
}, { timestamps: true });
```

■ C. Teacher Schema (`Teacher`)

Represents academic professors who configure sessions, start camera captures, and log attendance.

```
const TeacherSchema = new mongoose.Schema({
  fullName: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
```

```
  phone: { type: String },
  gender: { type: String },
  dob: { type: String },                          // Date of birth: YYYY-MM-DD
  qualification: { type: String },                // Academic degree (e.g. Ph.D)
  experience: { type: String },                   // Years of teaching experience
  department: { type: String, required: true },   // Department reference string
  primarySubject: { type: String },               // Default assigned subject name
  profilePhoto: { type: String },                 // Compressed Base64 profile thumbnail
  documents: [{ name: String, data: String }],    // Academic certifications or proof documents
  username: { type: String },
  role: { type: String, default: 'teacher' }
}, { timestamps: true });
```

■ D. Student Schema (`Student`)

Represents students who enroll their facial profiles and log onto their dashboards to view reports.

```
const StudentSchema = new mongoose.Schema({
  fullName: { type: String, required: true },
  email: { type: String, required: true },
  phone: { type: String },
  gender: { type: String },
  dob: { type: String },
  enrollmentNumber: { type: String, required: true, unique: true }, // University Roll Number
  department: { type: String },                   // Student department name
  course: { type: String },                       // Degree type (e.g., B.Tech CSE, BCA)
  semester: { type: String },                     // Current active semester (e.g., 4th Semeste
r)
  batchYear: { type: String },                    // Entry year (e.g., 2022)
  profilePhoto: { type: String },                 // Traditional registration photo
  username: { type: String },                     // Custom dashboard login username (Roll No)
  password: { type: String },                     // Student password string
  enrolledFace: { type: String },                 // Frontal facial capture base64 URL used by
Face AI
  lastEnrollmentUpdate: { type: Date },           // Face registration timestamp
  faceData: { type: String },                     // Extra face descriptor vector cache
  registrationDate: { type: Date, default: Date.now }
}, { timestamps: true });
```

■ E. Attendance Schema (`Attendance`)

Represents individual lecture logs that record active students, marking modes, and the corresponding photo proofs.

```
const AttendanceSchema = new mongoose.Schema({
  date: { type: String, required: true },         // Format: YYYY-MM-DD
  teacherId: { type: String, required: true },    // Reference to marking Teacher
  teacherName: { type: String },
  subject: { type: String },                      // Class subject name (e.g., Data Structures)
  semester: { type: String },                     // Target semester
  course: { type: String },                       // Mapped course (e.g., BCA) for admin filter
ing
  department: { type: String },                   // Mapped department name
  session: { type: String },                      // Lecture session identification (e.g. Lectu
re 1)
  attendance: { type: Object, required: true },   // Key-Value structure: { [studentId]: "Prese
nt" | "Absent" }
  proofPhoto: { type: String },                   // High-resolution watermarked group proof (B
ase64 JPEG)
  submissionDate: { type: String }                // Submission timestamp string
}, { timestamps: true });
```